
Chapter 16: Recovery System



Chapter 16: Recovery System

- ▶ **Failure Classification**
- ▶ **Storage Structure**
- ▶ **Recovery and Atomicity**
- ▶ **Recovery Algorithm**
- ▶ **Failure with Loss of Nonvolatile Storage**
- ▶ **Buffer Management**
- ▶ **Early Lock Release and Logical Undo Operations**



Chapter 16: Recovery System

- ▶ A computer system is *subject to failure* from *a variety of causes*: *disk crash, power outage, software error, a fire in the machine room*, even *sabotage*.
- ▶ The DB system must take actions to preserve the *atomicity* and *durability* properties.
 - ▶ An integral part of a database system is a *recovery scheme*, restoring the database to the consistent state that existed before the failure.
 - ▶ *High availability* must be provided: minimizing the time for which the DB is not usable after a failure.



Failure Classification

- ▶ We consider only the following *types of failure* in this chapter
 - ▶ **Transaction failure**
 - ▶ *Logical errors*: the transaction can not continue with its normal execution because of some *internal condition* (bad input, data not found, overflow, or resource limit exceeded).
 - ▶ *System errors*: the system has entered an *undesirable state* (e.g., deadlock), as a result of which a transaction cannot continue with its normal execution (can be *reexecuted at a later time*).



Failure Classification (Cont.)

- ▶ We consider only the following *types of failure* in this chapter
- ▶ **System crash**: a *hardware malfunction*, or a *bug* causes the loss of the content of *volatile storage*, and brings transaction processing to a *halt* (the content of *nonvolatile* storage remains *intact*, and is *not corrupted*).
 - ▶ *Fail-stop assumption*: *non-volatile* storage contents are assumed to *not be corrupted by system crash* (hardware errors and bugs in the software).
 - ▶ The fail-stop assumption is a *reasonable one*, because well-designed systems have numerous internal checks that bring the system to a halt when there is an error.



Failure Classification (Cont.)

- ▶ We consider only the following *types of failure* in this chapter
- ▶ **Disk failure:** a *head crash* or other failure during a data-transfer operation causes the *loss of the content* of a disk block (copies of the data on other disks, or archival backups on tertiary media, such as DVD or tapes, are used to recover from disk failure).



Recovery Algorithms

- ▶ To determine *how the system should recover from failures*, we need to
 - ▶ Identify the *failure modes* of those devices used for storing data.
 - ▶ Consider how these failure modes affect the *contents of the database*.



Recovery Algorithms (Cont.)

- ▶ We can then *propose algorithms to ensure database consistency and transaction atomicity* despite failures.
- ▶ These *recovery algorithms* have two parts:
 1. *Actions* taken *during normal transaction* processing to ensure *enough* information exists to recover from failures.
 2. *Actions* taken *after a failure* to recover the database contents to a state that ensures *atomicity*, *consistency* and *durability*.



Storage Structure

- ▶ ***Volatile storage***

- ▶ does not survive system crashes.
- ▶ E.g., main memory, cache memory.

- ▶ ***Nonvolatile storage***

- ▶ survives system crashes
- ▶ E.g., disk, tape, flash memory, non-volatile (battery backed up) RAM.
- ▶ but may still fail, losing data.

- ▶ ***Stable storage***

- ▶ No data loss.



Stable-Storage Implementation

- ▶ To implement **stable storage**, we need to *replicate the needed information in several nonvolatile storage media* (e.g., disk) with independent failure modes, and to update the information in a controlled manner to *ensure that failure during data transfer does not damage the needed information*.
- ▶ **RAID systems** guarantee that the *failure of a single disk (even during data transfer) will not result in loss of data*, but *cannot guard against data loss due to disasters* such as fires or flooding.



Stable-Storage Implementation (Cont.)

- ▶ How storage media can be *protected from failure during data transfer* ?
- ▶ **Block transfer** between memory and disk can result in
 - ▶ **Successful completion**: information arrived safely.
 - ▶ **Partial failure**: failure occurred in the midst of transfer and destination block has incorrect information.
 - ▶ **Total failure**: failure occurred sufficiently early and destination block remains intact.



Stable-Storage Implementation (Cont.)

- ▶ We demand if a *data-transfer failure* occurs, the system *detects it* and invokes a *recovery procedure* to *restore the block to a consistent state*.
- ▶ To do so, the system maintain *two physical blocks* for each logical database block in the mode of *mirrored disks* or *remote backup*.



Stable-Storage Implementation (Cont.)

- ▶ *An output operations is executed* as follow
 1. *Write* the information onto the *first physical block*.
 2. When the first write successfully completes, write the *same information* onto the *second physical block*.
 3. The output is *completed* only after the second write successfully completes.



Stable-Storage Implementation (Cont.)

- ▶ *Copies of a block may differ due to failure during output operation.* To recover from failure
 1. *First find inconsistent blocks:* comparing the two copies of every disk block.
 2. If either copy of *an inconsistent block is detected to have an error* (bad checksum), overwrite it by the other copy.
 3. If *both contain no detectable error, but they differ in content*, replaces the content of the first block with the value of the second.

Note: Comparing every corresponding pair of blocks is *expensive*. We can reduce the cost greatly by *keeping track of block writes that are in progress*, using a small amount of nonvolatile RAM.

Data Access

- ▶ As we know, the *DB system* resides *permanently* on nonvolatile storage and only *parts of the database* are *in memory* at any time.
- ▶ The DB is partitioned into *fixed-length storage units* called *blocks*, which are the units of data transfer.
 - ▶ The blocks residing on the disk are *physical blocks*.
 - ▶ The blocks residing temporarily in main memory are *buffer blocks*.
 - ▶ The area of memory where blocks reside temporarily is called the *disk buffer*.



Data Access (Cont.)

- ▶ *Block movements between disk and main memory* are initiated through the following two operations:
 - ▶ **input(B)** transfers the physical block B to main memory.
 - ▶ **output(B)** transfers the buffer block B to the disk, and replaces the appropriate physical block there.
- ▶ We assume, for simplicity, that *each data item fits in, and is stored inside, a single block.*

Data Access (Cont.)

- ▶ Each transaction T_i has its *private work-area* in which *local copies of all data items* accessed and updated by T_i are kept.
- ▶ This work area are *created* when the transaction is *initiated*, and *removed* when the transaction either *commits* or *aborts*.
- ▶ Each data item X kept in the work area of transaction T_i is denoted by x_i .



Data Access (Cont.)

- ▶ *Transferring data items* between *system buffer blocks* and its *private work-area* done by
 - ▶ **read(X)** assigns the value of data item X to the local variable x_i .
 - ▶ If block B_X on which X resides is not in main memory, it issues $\text{input}(B_X)$.
 - ▶ It assigns to x_i the value of X from the buffer block.

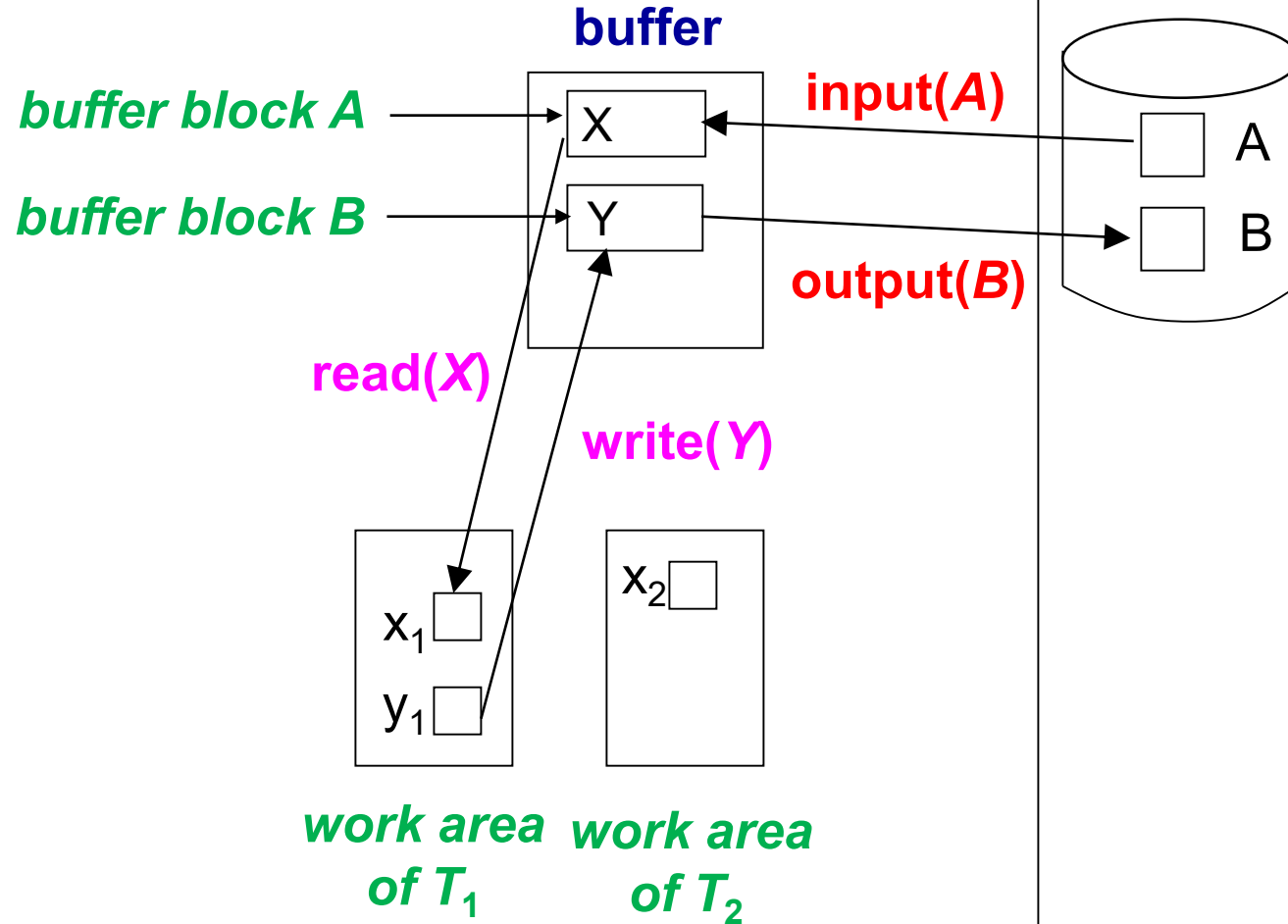


Data Access (Cont.)

- ▶ *Transferring data items* between *system buffer blocks* and its *private work-area* done by
 - ▶ **write(X)** assigns the value of local variable x_i to data item X in the buffer block.
 - ▶ If block B_X on which X resides is not in main memory, it issues $\text{input}(B_X)$.
 - ▶ It assigns the value of x_i to X in buffer B_X .



Example of Data Access



memory

disk



Data Access (Cont.)

- ▶ A *buffer block* is *eventually written out to the disk* either because
 - ▶ the buffer manager needs the *memory space* for other purposes.
 - ▶ the DB system wishes to *reflect* the change to B on the disk.
- ▶ If it issues an **output(B)**, the DB system performs a *force-output* of buffer B .

Data Access (Cont.)

- ▶ When a transaction needs to *access a data item X for the first time*, it must execute **read(X)**. The system then performs *all updates to X on x_i* .
- ▶ At any point during its execution, a transaction may execute **write(X)** to *reflect the change to X in the database itself*
- ▶ **Write(X)** must certainly be done after the final write to X .



Data Access (Cont.)

- ▶ The **output(B_X)** operation does not need to *take effect immediately after write(X) is executed*, since the block B_X may contain other data items that are still being accessed.
- ▶ If the system crashes *after* the write(X) operation was executed but *before* output(B_X) was executed, the new value of X is *never* written to disk (*lost*).

write(X) ----- system crashes ----- output(B_X)

- ▶ The DB system executes *extra actions* to ensure that *updates performed by committed transactions are not lost even if there is a system crash*.



Recovery and Atomicity

- ▶ Consider banking system and a transaction T_i : transfer \$50 from account A to account B , with initial values $A = \$1000$ and $B = \$2000$.
- ▶ Suppose that a system crash occurred *after* $\text{output}(B_A)$ has taken place, but *before* $\text{output}(B_B)$ was executed, where B denote the buffer block.
- ▶ With system restarted, $A = \$950$ and $B = \$2000$, which is clearly inconsistent with the atomicity requirement for transaction T_i .
- ▶ There is *no way to find out* what blocks had been output by examining the database state, and what had not *before the crash*. There are many possible situations.

Recovery and Atomicity (Cont.)

- ▶ Our goal is to *perform either all or no database modifications made by T_i*
- ▶ However, if T_i performed multiple modifications, and a failure may occur after *some* of these modifications have been made, with the *other* of them not made.
- ▶ To ensure *atomicity* despite failures, we *first output information describing the modifications to stable storage*, before modifying the database itself
 - ▶ help us ensure that *all modifications* performed by committed transactions are reflected in the database.
 - ▶ also help us ensure that *no modifications* made by an aborted transaction persist in the database.

Log-Based Recovery

- ▶ The most widely used structure for *recording database modifications* is the **log** (*a sequence of log records*), recording all the update activities in the database.
- ▶ An *update log record* describes *a single database write*
 - ▶ *Transaction identifier* -- *unique identifier of the transaction* that performed the write operation.
 - ▶ *Data-item identifier* -- *unique identifier of the data item written*. Typically, it is the *location* on disk of the data item, consisting of the *block identifier* of the block on which the data item resides, and an *offset* within the block.



Log-Based Recovery (Cont.)

- ▶ The most widely used structure for *recording database modifications* is the **log** (*a sequence of log records*), recording all the update activities in the database.
- ▶ An *update log record* describes *a single database write*
 - ▶ *Old value* -- value of the data item *prior to the write*
 - ▶ *New value* -- value that the data item will have *after the write*



Log Records Example

- ▶ When transaction T_i starts, it registers itself by writing a **< T_i start>** log record
- ▶ Before T_i executes **write(X_j)**, a log record
< T_i , X_j , V_1 , V_2 >
is written. where V_1 is the value of data item X_j before the write (**old value**), and V_2 is the value to be written to X_j (**new value**).
- ▶ When T_i finishes its last statement, the log record **< T_i commit>** is written.
- ▶ When T_i aborts, the log record **< T_i abort>** is written.



Log-Based Recovery (Cont.)

- ▶ Whenever a transaction *performs a write*, it is essential that the *log record* for that write be created, and added to the *log*, before the database is modified.
- ▶ Once a *log record exists*, we can *output the modification* to the database if that is desirable. Also, we have the ability to *undo a modification* that has already been output to the database. We undo it by using the *old-value field* in log records.
- ▶ For log records to be useful for *recovery* from system and disk failures, the *log must reside in stable storage*.



Database Modification

- ▶ The *log records* allow
 - ▶ system to *undo changes* made by a transaction in the event that the transaction must be *aborted*
 - ▶ system to *redo changes* made by a transaction if the transaction has *committed* but the *system crashed* before those changes could be stored in the database on disk
- ▶ The steps a transaction takes in *modifying a data item*
 1. The transaction performs some *computations* in its own *private part* of main memory.
 2. The transaction *modifies* the data block in the *disk buffer* in main memory holding the data item.
 3. The database system executes the *output* operation that writes the data block to *disk*.

Database Modification (Cont.)

- ▶ We say a transaction *modifies the database* if it performs an *update* on a *disk buffer*, or on the *disk itself*.
- ▶ If a transaction doesn't modify database until it has committed, it's said to use *deferred-modification* technique
 - ▶ Extra overhead is that transactions need *to make local copies of all updated data items*.
 - ▶ further, if a transaction reads a data item that it has updated, it must *read the value from its local copy*.
- ▶ If database modifications occur while the *transaction is still active*, the transaction is said to use the *immediate-modification* technique.



Database Modification (Cont.)

- ▶ A *recovery algorithm* must take into account a variety of *factors*, including
 - ▶ The *possibility* that a transaction may have *committed* although some of its database *modifications* exist only in the *disk buffer in main memory* and not in the database on disk.
 - ▶ The *possibility* that a transaction may have *modified the database* while in the *active state* and may need to *abort as a result of a subsequent failure*.



Database Modification (Cont.)

- ▶ Because *all database modifications must be preceded by the creation of a log record*, the system has available both the *old value* prior to the modification of the data item and the *new value* that is to be written for the data item.
- ▶ This allows the system to perform *undo* and *redo* operations as appropriate
 - ▶ *Undo* using a *log record* sets the data item specified in the log record to the old value.
 - ▶ *Redo* using a *log record* sets the data item specified in the log record to the new value.



Concurrency Control and Recovery

- ▶ If the *concurrency control scheme* allows *a data item X* that *has been modified* by transactions T_1 to be *further modified* by T_2 before T_1 commits, then *undoing the effects* of T_1 by restoring the *old value of X* (before T_1 updated X) would also undo the effects of T_2 . It is not desirable.
- ▶ To avoid such situations, *recovery algorithms* usually require that if a data item has been modified by a transaction, *no other transaction can modify the data item until the first transaction commits or aborts*.



Concurrency Control and Recovery

- ▶ The requirement can be ensured by *acquiring an exclusive lock* on any updated data item and holding the lock *until the transaction commits* (*strict two-phase locking*).
- ▶ *Snapshot-isolation* and *validation* based concurrency - control techniques also *acquire exclusive locks* on data items during *validation* (before modifying), and hold the locks until *the transaction is committed*.



Transaction Commit

- ▶ A transaction is said to have **committed** when **its commit log record is output to stable storage** (at that point, all previous log records of the transaction must have already been output to stable storage).
- ▶ There is **enough information** ensure that even if there is a system crash, the updates of the transaction can be redone.



Transaction Commit

- ▶ If a *system crash* occurs before a log record $\langle T_i \text{ commit} \rangle$ is output to stable storage, transaction T_i will be *rolled back*.
- ▶ Thus, the *output* of the block containing the commit log record is the *single atomic action* that results in a transaction getting committed.
- ▶ **Note:** With most log-based recovery techniques, blocks containing the data items modified by a transaction do *not have to be output* to stable storage *when the transaction commits*, but can be *output some time later*.



Using the Log to Redo and Undo Transactions

- ▶ We now provide an overview of *how the log can be used to recover from a system crash, and to roll back transactions* during normal operation.
- ▶ Consider the two fund transfer transactions:

T_0 : **read**(A);
 $A := A - 50$;
 write(A);
 read(B);
 $B := B + 50$;
 write(B).

T_1 : **read**(C);
 $C := C - 100$;
 write(C).

withdraws \$100 from account C

transfers \$50 from account A to account B



Using the Log to Redo and Undo Transactions

T_0 : **read**(A);
 $A := A - 50$;
 write(A);
 read(B);
 $B := B + 50$;
 write(B).

T_1 : **read**(C);
 $C := C - 100$;
 write(C).

$\langle T_0 \text{ start} \rangle$
 $\langle T_0, A, 1000, 950 \rangle$
 $\langle T_0, B, 2000, 2050 \rangle$
 $\langle T_0 \text{ commit} \rangle$
 $\langle T_1 \text{ start} \rangle$
 $\langle T_1, C, 700, 600 \rangle$
 $\langle T_1 \text{ commit} \rangle$

**Portion of the system log
corresponding to T_0 and T_1**



Using the Log to Redo and Undo Transactions

Log	Database
$\langle T_0 \text{ start} \rangle$	
<u>$\langle T_0, A, 1000, 950 \rangle$</u>	
<u>$\langle T_0, B, 2000, 2050 \rangle$</u>	
	$A = 950$
	$B = 2050$
$\langle T_0 \text{ commit} \rangle$	
$\langle T_1 \text{ start} \rangle$	
<u>$\langle T_1, C, 700, 600 \rangle$</u>	
	$C = 600$
$\langle T_1 \text{ commit} \rangle$	

This is one possible order in which the **actual outputs** took place in both the DB system and the log as a result of the execution of T_0 and T_1 .



Undo and Redo Operations

- ▶ Using the *log*, the system can *handle any failure* that does not result in the loss of information in nonvolatile storage.
- ▶ The *recovery scheme* uses two recovery procedures
 - ▶ *Undo* of a log record $\langle T_i, X, V_1, V_2 \rangle$ writes the *old value* V_1 to X .
 - ▶ *Redo* of a log record $\langle T_i, X, V_1, V_2 \rangle$ writes the *new value* V_2 to X .
- ▶ Both these procedures make use of the log to find the set of data items updated by each transaction T_i , and their respective old and new values.



Undo and Redo Operations (Cont.)

▶ *Undo and Redo of Transactions*

- ▶ *redo(T_i)* sets the value of all data items updated by T_i to the *new values*
 - ▶ The *order* in which updates are carried out by redo is important (*must be same as the original operations*).
 - ▶ Most recovery algorithms *do not perform redo of each transaction separately*; instead they perform *a single scan of the log*, during which redo actions are performed for *each log record* as it is encountered.
 - ▶ This approach *ensures the order of updates is preserved*, and is more efficient, since *the log needs to be read only once* overall, instead of once per transaction.

Undo and Redo Operations (Cont.)

► *Undo and Redo of Transactions*

- **undo(T_i)** restores the value of all data items updated by T_i to their *old values*, going backwards *from the last log record* for T_i .
 - Each time a data item X is restored to its *old value* V , and a special *redo-only log record* $\langle T_i, X, V_1 \rangle$ is written out (*no old value*), since they do not need to contain the new-value of the updated data item.
 - When the *undo operation* for transaction T_i *completes*, it writes a $\langle T_i, \text{abort} \rangle$ log record, indicating that the undo has completed.
 - The *order* in which undo operations are performed is important.

Undo and Redo on Recovering from Failure

▶ *When recovering after failure*

- ▶ Transaction T_i needs to be *undone* if the log
 - ▶ contains the record $\langle T_i \text{ start} \rangle$.
 - ▶ but does not contain either the record $\langle T_i \text{ commit} \rangle$ or $\langle T_i \text{ abort} \rangle$.
- ▶ Transaction T_i needs to be *redone* if the log
 - ▶ contains the record $\langle T_i \text{ start} \rangle$.
 - ▶ and contains the record $\langle T_i \text{ commit} \rangle$ or $\langle T_i \text{ abort} \rangle$.



Undo and Redo on Recovering from Failure (Cont.)

- ▶ *Why redo T_i if the record $\langle T_i \text{ abort} \rangle$ is in the log*
 - ▶ If $\langle T_i \text{ abort} \rangle$ is in the log, so are the *redo-only* records written by the *undo operation*.
 - ▶ Thus, the *end result* will be to *undo* T_i 's modifications in this case.
 - ▶ The slight redundancy *simplifies the recovery algorithm* and enables faster overall recovery time.

Recovery Example

T_0 : **read**(A);
 $A := A - 50$;
write(A);
read(B);
 $B := B + 50$;
write(B).

T_1 : **read**(C);
 $C := C - 100$;
write(C).

$\langle T_0 \text{ start} \rangle$
 $\langle T_0, A, 1000, 950 \rangle$
 $\langle T_0, B, 2000, 2050 \rangle$
 $\langle T_0 \text{ commit} \rangle$
 $\langle T_1 \text{ start} \rangle$
 $\langle T_1, C, 700, 600 \rangle$
 $\langle T_1 \text{ commit} \rangle$

- (a) *crash occurs just after the log record for write(B)*

$\langle T_0 \text{ start} \rangle$

$\langle T_0 \text{ start} \rangle$ is found

$\langle T_0, A, 1000, 950 \rangle$

$\langle T_0, B, 2000, 2050 \rangle$

No $\langle T_0 \text{ commit} \rangle$ or $\langle T_0 \text{ abort} \rangle$

- **undo** (T_0): B is restored to 2000 and A to 1000, and log records $\langle T_0, B, 2000 \rangle$ (*redo-only*), $\langle T_0, A, 1000 \rangle$ (*redo-only*), $\langle T_0, \text{abort} \rangle$ are written out.

Recovery Example (Cont.)

► (b) *Crash occur after the log record for write(C)*

$\langle T_0 \text{ start} \rangle$

$\langle T_0, A, 1000, 950 \rangle$

$\langle T_0, B, 2000, 2050 \rangle$

$\langle T_0 \text{ commit} \rangle$

$\langle T_1 \text{ start} \rangle$

$\langle T_1, C, 700, 600 \rangle$

Both $\langle T_0 \text{ start} \rangle$ and $\langle T_0 \text{ commit} \rangle$
are found

$\langle T_1 \text{ start} \rangle$ is found

Not $\langle T_1 \text{ commit} \rangle$ or $\langle T_1 \text{ abort} \rangle$

- **redo (T_0)** and **undo (T_1)**: A and B are set to 950 and 2050 and C is restored to 700. Log records $\langle T_1, C, 700 \rangle$ (*read-only*), $\langle T_1, \text{abort} \rangle$ are written out.



Recovery Example (Cont.)

- ▶ (c) *crash occurs just after the log record $\langle T_1 \text{ commit} \rangle$*

$\langle T_0 \text{ start} \rangle$

$\langle T_0, A, 1000, 950 \rangle$

$\langle T_0, B, 2000, 2050 \rangle$

$\langle T_0 \text{ commit} \rangle$

$\langle T_1 \text{ start} \rangle$

$\langle T_1, C, 700, 600 \rangle$

$\langle T_1 \text{ commit} \rangle$

Both $\langle T_0 \text{ start} \rangle$ and $\langle T_0 \text{ commit} \rangle$
are found

Both $\langle T_1 \text{ start} \rangle$ and $\langle T_1 \text{ commit} \rangle$
are found

- ▶ **redo (T_0) and redo (T_1):** A and B are set to 950 and 2050 respectively. Then C is set to 600.



Checkpoints

- ▶ If we need to *search the entire log* to determine the information those transactions that need to be *redone* and those that need to be *undone*, the problems are
 1. the search process is *time-consuming*.
 2. we might *unnecessarily redo transactions* which have already *written their updates into the database*.
- ▶ *Checkpoints*: reduce these types of overhead.

Checkpoints (Cont.)

- ▶ We describe below *a simple checkpoint scheme* that
 1. *does not permit any updates* to be performed while the *checkpoint operation is in progress*
 2. *outputs all modified buffer blocks to disk* when the checkpoint is performed.



Checkpoints (Cont.)

► *A checkpoint is performed as follows*

1. *Output onto stable storage all log records currently residing in main memory*
2. *Output to the disk all modified buffer blocks*
3. *Output onto stable storage a log record of the form $\langle \text{checkpoint } L \rangle$, where L is a list of transactions active at the time of the checkpoint.*



Checkpoints (Cont.)

- ▶ The presence of a **<checkpoint L >** record in the log allows the system to *streamline its recovery procedure*
- ▶ Consider a transaction T_i that completed *prior to the checkpoint*
- ▶ For such a transaction, the **< T_i commit>** record (or **< T_i abort>** record) appears in the log *before the <checkpoint> record*
- ▶ *Any database modifications made by T_i must have been written to the database*
- ▶ Thus, at recovery time, there is *no need to perform a redo operation on T_i*

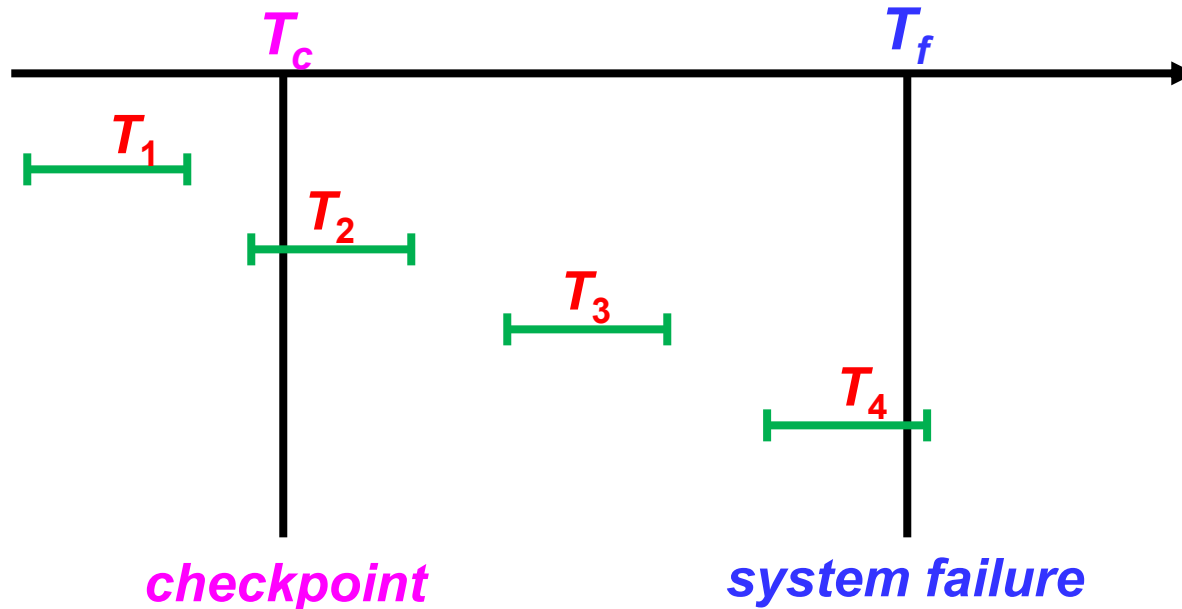


Checkpoints (Cont.)

- ▶ After a *system crash* has occurred, the system examines the log to find the *last <checkpoint L> record*.
- ▶ The *redo* or *undo* operations need to be applied only to *transactions in L*, and to *all transactions* (denoted as *T*) *that started execution after the <checkpoint L>*
 - ▶ For all transactions T_k in *T* that have *no <T_k commit> record* or *<T_k abort> record* in the log, execute *undo(T_k)*
 - ▶ For all transactions T_k in *T* that either the *record <T_k commit>* or the *record <T_k abort>* appears in the log, execute *redo(T_k)*



Example of Checkpoints



- T_1 can be *ignored* (updates already output to disk due to checkpoint)
- T_2 and T_3 *redone*
- T_4 *undone*



Checkpoints (Cont.)

- ▶ Transactions must **NOT** perform any updates to *buffer blocks* or to the *log during checkpointing*, since transaction processing has to halt while a checkpoint is in progress.
- ▶ We will describe some other checkpoint schemes which *allow updates during checkpointing* later (*fuzzy checkpoint*).



Recovery Algorithm

- ▶ We have *identified* transactions that need to be *redone* and those that need to be *undone*, but we have not given *a precise algorithm* for performing these actions.
- ▶ We now present the *full recovery algorithm* using *log records* for *recovery from transaction failure* and a combination of the *most recent checkpoint and log records* to *recover from a system crash*.
- ▶ The recovery algorithm requires that a data item that has been updated by an *uncommitted transaction cannot be modified by any other transaction*, until the first transaction has either *committed* or *aborted*.



Transaction Rollback

- ▶ *Transaction rollback during normal operation* (let T_i be the transaction to be rolled back).
 - ▶ The log is *scanned backwards from the end*, and for each log record of T_i of the form $\langle T_i, X_j, V_1, V_2 \rangle$
 - ▶ write V_1 to data item X_j (undo);
 - ▶ write a *redo-only* log record $\langle T_i, X_j, V_1 \rangle$ to the log (*compensation log records* and do not need undo information, since we *never need to undo such an undo operation*).
 - ▶ Once the record $\langle T_i, \text{start} \rangle$ is found, stop the scan and write record $\langle T_i, \text{abort} \rangle$ to the log.
- ▶ **[Note]** *Every update action*, including actions taken to *restore data items to their old value*, have now been recorded in the log.

Recovery After a System Crash

- ▶ *Recovery from failure* take place in two phases, when the *database system is restarted after a crash*
- ▶ *Redo Phase*: System *replays updates of all transactions* by scanning the log forward from the last checkpoint
 - ▶ *Log records that were replayed* include: 1) *log records for transactions* that were *rolled back before system crash*; 2) *log records for transactions* that had *not committed when the system crash occurred*.

Recovery After a System Crash

- ▶ *Recovery from failure* take place in two phases, when the *database system is restarted after a crash*
- ▶ *Redo Phase*: System *replays updates of all transactions* by scanning the log forward from the last checkpoint
 - ▶ This phase also determines *all transactions that were incomplete at the time of the crash*, and must therefore be rolled back.
 - ▶ *Such incomplete transactions* would either have been *active at the time of the checkpoint*, and thus would appear in the *transaction list* in the checkpoint record, or would have *started later*, without a *<T_i abort>* nor a *<T_i commit>* record in the log.

Recovery After a System Crash (Cont.)

- ▶ The specific steps taken while *scanning the log forward* are as follows:
 1. Set the *undo-list* to the list L in the *<checkpoint L >* log record
 2. Whenever a *normal log* record $\langle T_i, X_j, V_1, V_2 \rangle$ or $\langle T_i, X_j, V_2 \rangle$ (*redo-only*) is found, *redo* it by writing V_2 to X_j
 3. Whenever a log record *< T_i start>* is found, *add T_i to undo-list*
 4. Whenever a log record *< T_i commit>* or *< T_i abort>* is found, *remove T_i from undo-list*



Recovery After a System Crash (Cont.)

- ▶ *Recovery from failure* take place in two phases, when the *database system is restarted after a crash*
- ▶ **Undo Phase**: By scanning the log backward from the end, the system *rolls back all transactions in the undo-list*
- ▶ The specific steps taken while *scanning the log backward* are as follows:
 1. Whenever a log record $\langle T_i, X_j, V_1, V_2 \rangle$ for a transaction *T_i is found in undo-list*, it perform undo actions just like the *rollback of a failed transaction*:
 - a) *perform undo by writing V_1 to X_j* ;
 - b) *write a log record $\langle T_i, X_j, V_1 \rangle$ (redo-only)*

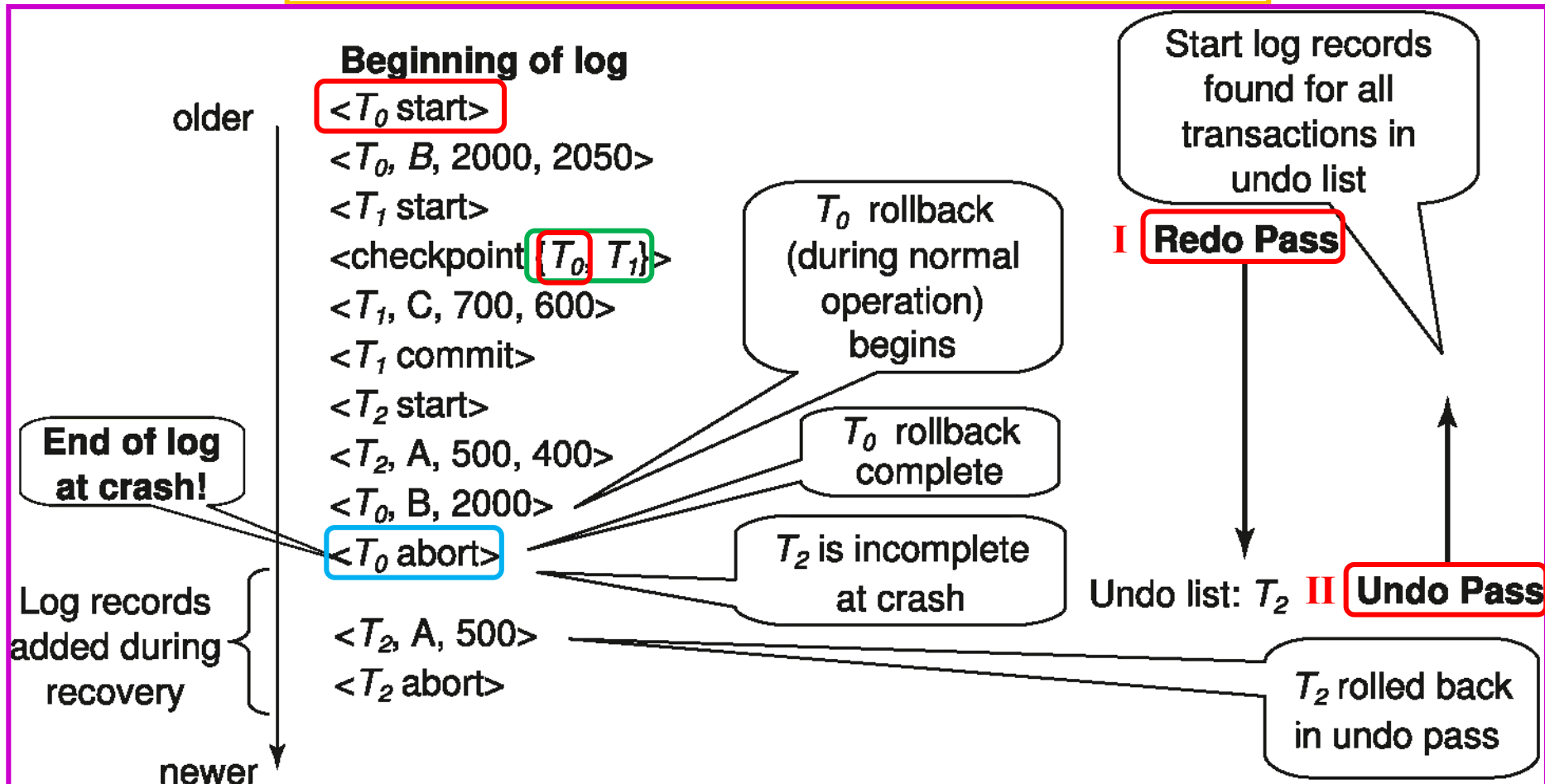


Recovery After a System Crash (Cont.)

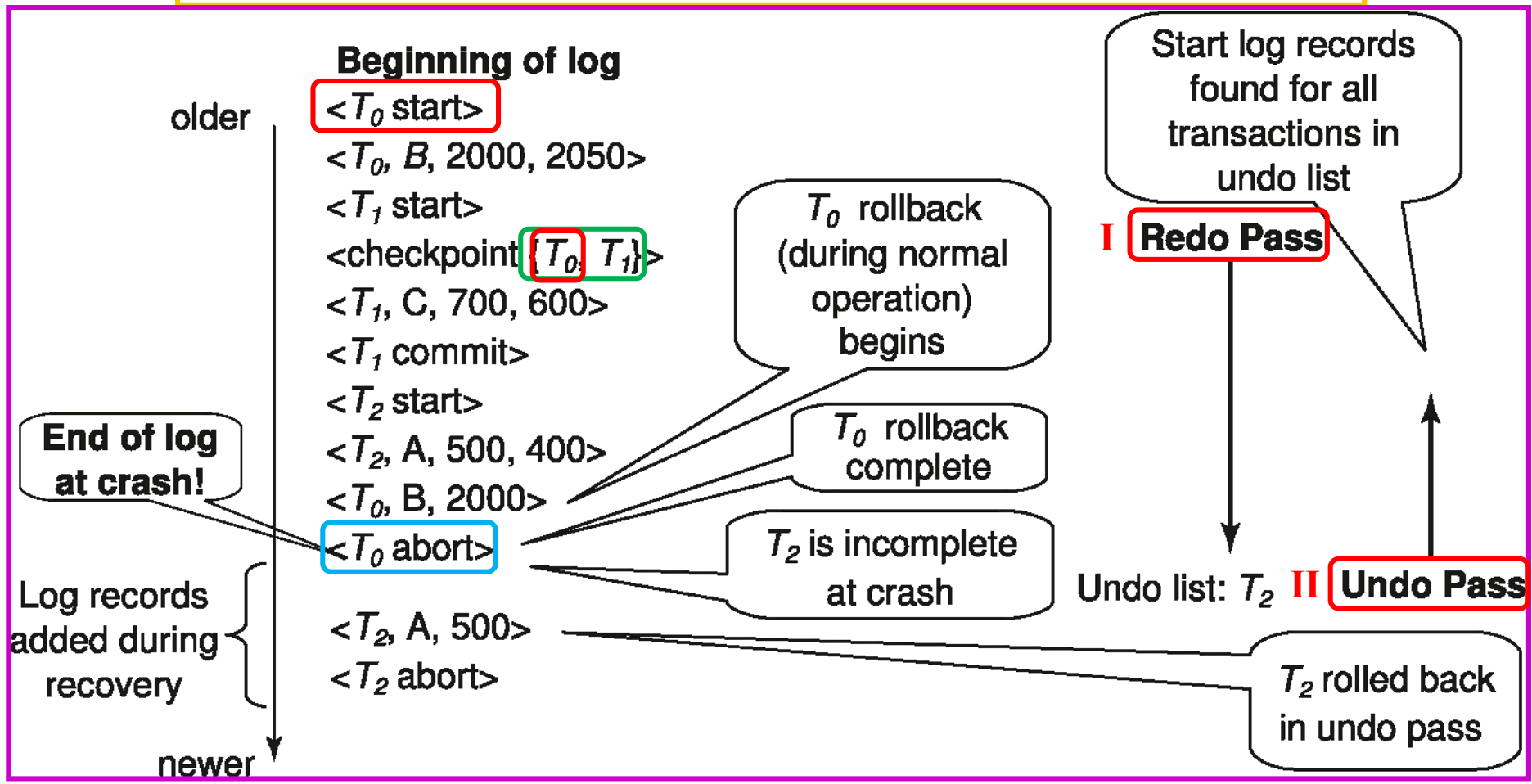
2. Whenever a log record $\langle T_i \text{ start} \rangle$ for a transaction T_i is *found in undo-list*:
 - a) *write a log record $\langle T_i \text{ abort} \rangle$;*
 - b) *remove T_i from undo-list.*
 2. The *undo phase terminates* when *undo-list is empty*: the system has found $\langle T_i \text{ start} \rangle$ log records for all transactions that were initially in undo-list.
- After undo phase completes, *normal transaction processing* can resume.



- ▶ **Redo Phase:** system performs a *redo of all operations* after *last checkpoint record*
 - ▶ The list **undo-list** initially contains T_0 and T_1
 - ▶ T_1 is *removed* first when $\langle T_1 \text{ commit} \rangle$ is found
 - ▶ T_2 is *added* when its $\langle T_2 \text{ start} \rangle$ is found
 - ▶ T_0 is *removed* from **undo-list** when $\langle T_0 \text{ abort} \rangle$ record is found, leaving only T_2 in **undo-list**



- ▶ **Undo Phase:** scans the log *backwards from the end*
 - ▶ When $\langle T_2, A, 500, 400 \rangle$ found, the old value of $A(500)$ is restored, and a *redo-only* log record $\langle T_2, A, 500 \rangle$ written to the log.
 - ▶ When the $\langle T_2, \text{start} \rangle$ is found, an abort record $\langle T_2, \text{abort} \rangle$ is added for T_2 .
 - ▶ Since *undo-list* contains no more transactions, the **undo phase** terminates (**recovery completed**).



Recovery After a System Crash (Cont.)

- ▶ Observe that the *redo phase replays every log record* since the most recent checkpoint record.
- ▶ This phase of restart recovery *repeats all the update actions* that were executed *after the checkpoint*, and whose *log records reached the stable log*.
- ▶ The actions include *actions of incomplete transactions* and the *actions carried out to rollback failed transactions*. The actions are *repeated in the same order* in which they were originally carried out.
- ▶ This process is called *repeating history*. Although it may appear wasteful, repeating history even for failed transactions *simplifies recovery schemes*.



Early Lock Release and Logical Undo Operations

- ▶ Any index used in processing a transaction, such as a B⁺-tree, can be treated as *normal data*.
- ▶ But to *increase concurrency*, we can use the B⁺-tree concurrency control algorithm to allow locks to be *released early in a non-two-phase manner*.
- ▶ As a result of *early lock release*, it is possible that:
 - ▶ A value in a B⁺-tree node is updated by *one transaction T_1 , which inserts an entry (V_1, R_1)* ;
 - ▶ And subsequently by *another transaction T_2 , which inserts an entry (V_2, R_2) in the same node, moving the entry (V_1, R_1) even before T_1 completes execution.*



Early Lock Release and Logical Undo Operations

- ▶ As a result of *early lock release*, it is possible that
 - ▶ At this point, *transaction T_1 cannot be undone* by replacing the contents of the node with the old value, *since that would also undo the insert performed by T_2*
 - ▶ Transaction T_2 *may still commit* (or may have already committed).
- ▶ In this example, the only way to undo the effect of transaction T_1 is to *execute a corresponding delete operation*.



Logical Operations

- ▶ The *insertion and deletion operations* are examples of a class of operations that *require logical undo operations since they release locks early*; we call such operations *logical operations*.
- ▶ Such *early lock release* is important not only for indices, but also for operations on other *system data structures* that are accessed and updated very frequently
 - ▶ Tracking the *blocks containing records of a relation*, the *free space in a block*, and the *free blocks in a database*.
 - ▶ If locks were not released early after performing operations on such data structures, transactions would *tend to run serially*, affecting system performance.



Logical Operations

- ▶ The theory of *conflict serializability* has been extended to operations (based on what operations conflict with what other operations).
- ▶ E.g. on a B⁺-tree, two insert operations *inserting different key values*, they *do not conflict*, even if they both update overlapping areas of the same index page.
- ▶ However, insert and delete operations *conflict with* other insert and delete operations, as well as with read operations, if they *use the same key value*.



Logical Operations (Cont.)

- ▶ Operations acquire *lower-level locks* while they execute, but release them when they complete
- ▶ For example, while an *insert operation* is being performed *on a B+-tree page*
 - ▶ A *short-term lock* is obtained on the page, allowing *entries in the page to be shifted during the insert*
 - ▶ The *short-term lock* is *released as soon as the page has been updated*.
 - ▶ Such *early lock release allows a second insert* to execute on the same page.



Logical Operations (Cont.)

- ▶ Operations acquire *lower-level locks* while they execute, but release them when they complete
- ▶ For example, while an *insert operation* is being performed *on a B+-tree page*
 - ▶ However, each transaction *must obtain a lock on the key values being inserted or deleted*, and retain it in *a two-phase manner*, to prevent a concurrent transaction from executing a conflicting read, insert or delete operation on the same key value.



Logical Operations (Cont.)

- ▶ Operations acquire *lower-level locks* while they execute, but release them when they complete
- ▶ For example, while an *insert operation* is being performed *on a B+-tree page*
 - ▶ Once the lower-level lock is released, the operation *cannot be undone by using the old values of updated data items*, and must *instead be undone by executing a compensating operation*; such an operation is called a *logical undo operation*.



Logical Undo Log Records

- ▶ *To allow logical undo of operations*

- ▶ Before an operation is performed to modify an index, the transaction *creates a log record* $\langle T_i, O_j, \text{operation-begin} \rangle$, where O_j is a unique identifier for the operation instance.
- ▶ While the system is executing the operation, it *creates update log records in the normal fashion for all updates performed by the operation*.
 - ▶ Thus, the usual *old-value* and *new-value* information is *written out for each update performed* by the operation.
 - ▶ The *old-value* is required in case the transaction *needs to be rolled back before the operation completes*.



Logical Undo Log Records (Cont.)

- ▶ *To allow logical undo of operations*
 - ▶ When the operation finishes, it *writes an operation-end log record of the form $\langle T_i, O_j, \text{operation-end}, U \rangle$* , where the *U* denotes *undo information*.
 - ▶ If the operation *insert an entry in a B^+ -tree*, the *undo information U* would
 - ▶ *indicate that a deletion operation is to be performed.*
 - ▶ *identify the B^+ -tree and what entry to delete from the tree.*



Logical Undo Log Records (Cont.)

- ▶ Such logging of information about operations is called *logical logging*.
- ▶ In contrast, *logging of old-value and new-value information* is called *physical logging*, and the corresponding log records are called *physical log records*.



Logical Undo Log Records (Cont.)

- ▶ **Note:** in the above scheme, *logical logging* is used only *for undo, not for redo*; *redo operations* are performed exclusively using *physical log record*.
- ▶ This is because the *state of the database* after a system failure may reflect *some updates of an operation* and *not other operations*, depending on what buffer blocks had been written to disk before the failure.
- ▶ *Data structures* such as B⁺-trees would *not be in a consistent state*, and *neither logical redo nor logical undo operations* can be performed on an inconsistent data structure.

Logical Undo Log Records (Cont.)

- ▶ To perform redo or undo, the database state on disk must be *operation consistent*, that is, *it should not have partial effects of any operation*.
- ▶ However, *before the logical undo operation is performed*, the *physical redo* processing in the *redo phase* of the recovery scheme, along with *undo* processing using *physical log records* ensures that *the parts of the database accessed by a logical undo operation are in an operation consistent state*.

Logical Undo Log Records (Cont.)

- ▶ An operation is said to be *idempotent* (幂等) if *executing it several times in a row gives the same result* as *executing it once*.
- ▶ If an operation is not idempotent (such as inserting an entry into a B⁺-tree), the *recovery algorithm* must therefore *make sure that the operation that has already been performed is not performed again*.
- ▶ On the other hand, *a physical log record is idempotent*, since the corresponding data item would have the *same value* regardless of whether the logged update is executed one or multiple times.



Transaction Rollback With Logical Undo

- ▶ When *rolling back a transaction T_i* , the *log is scanned backwards*, and log records corresponding to T_i are processed as follows:
 1. *Physical log records* encountered during the scan are *handled as described earlier*, except those that are skipped as described shortly. *Incomplete logical operations are undone using the physical log records generated by the operation.*



Transaction Rollback With Logical Undo

- ▶ When *rolling back a transaction T_i* , the *log is scanned backwards*, and log records corresponding to T_i are processed as follows:
 2. *Completed logical operations, identified by operation-end records*, are *rolled back differently*. Whenever the system finds a log record $\langle T_i, O_j, \text{operation-end}, U \rangle$,
 - a) It rolls back the operation by *using the undo information U* in the log record. It logs the updates performed during the rollback of the operation *just as the operation was first executed*. At the end of the operation rollback, the database system *generates a log record $\langle T_i, O_j, \text{operation-abort} \rangle$* , instead of $\langle T_i, O_j, \text{operation-end}, U \rangle$

Transaction Rollback With Logical Undo

- ▶ When *rolling back a transaction T_i* , the *log is scanned backwards*, and log records corresponding to T_i are processed as follows:
 2. *Completed logical operations, identified by operation-end records*, are *rolled back differently*. Whenever the system finds a log record $\langle T_i, O_j, \text{operation-end}, U \rangle$,
 - b) As the *backward scan* of the log continues, the system *skips* all log records of T_i until it finds the log record $\langle T_i, O_j, \text{operation-begin} \rangle$. After it finds the *operation-begin log record*, it processes *log records of T_i* in the *normal manner* again.

Transaction Rollback With Logical Undo

- ▶ When *rolling back a transaction T_i* , the *log is scanned backwards*, and log records corresponding to T_i are processed as follows:
 2. *Completed logical operations, identified by operation-end records*, are *rolled back differently*. Whenever the system finds a log record $\langle T_i, O_j, \text{operation-end}, U \rangle$,
 - ▶ **Note 1:** The system logs *physical undo information* for the *updates performed during rollback*, instead of using a redo-only compensation log records. It is because *a crash may occur* while a *logical undo is in progress*, and on recovery the system has to complete the logical undo; i.e., restart recovery will *undo the partial effects of the earlier undo* (using the physical undo information), and then *perform the logical undo again*.

Transaction Rollback With Logical Undo

- ▶ When *rolling back a transaction T_i* , the *log is scanned backwards*, and log records corresponding to T_i are processed as follows:
 2. *Completed logical operations, identified by operation-end records*, are *rolled back differently*. Whenever the system finds a log record *$\langle T_i, O_j, \text{operation-end}, U \rangle$* ,
 - ▶ **Note 2:** *Skipping over physical log records* when the *operation-end log record* is found during rollback ensures that the *old values in the physical log record* are *not used for rollback*, once the operation completes.



Transaction Rollback With Logical Undo

- ▶ When *rolling back a transaction T_i* , the *log is scanned backwards*, and log records corresponding to T_i are processed as follows:
 3. If the system finds a record $\langle T_i, O_j, \text{operation-abort} \rangle$, it *skips all preceding records* (including the operation-end record for O_j) until it finds the record $\langle T_i, O_j, \text{operation-begin} \rangle$.
- ▶ **Note:** An *operation-abort log record* would be found only if a transaction that is *being rolled back* had been *partially rolled back earlier*. Recall that *logical operations may not be idempotent*, and hence a *logical undo operation* must *not be performed multiple times*.

Transaction Rollback With Logical Undo

- ▶ When *rolling back a transaction T_i* , the *log is scanned backwards*, and log records corresponding to T_i are processed as follows:
 3. If the system finds a record *$\langle T_i, O_j, \text{operation-abort} \rangle$* , it *skips all preceding records* (including the operation-end record for O_j) until it finds the record *$\langle T_i, O_j, \text{operation-begin} \rangle$* .
- ▶ **Note:** These *preceding log records* must be *skipped to prevent multiple rollback of the same operation*, in case there had been a crash during an earlier rollback, and the transaction had already been partly rolled back.

Transaction Rollback With Logical Undo

- ▶ When *rolling back a transaction T_i* , the *log is scanned backwards*, and log records corresponding to T_i are processed as follows:
 3. As before, when the *$\langle T_i, \text{start} \rangle$* log record has been found, the transaction *rollback is complete*, and the system adds a record *$\langle T_i, \text{abort} \rangle$* to the log

Transaction Rollback With Logical Undo

- ▶ If a failure occurs while *a logical operation is in progress*, the *operation-end log record* for the operation *will NOT be found* when the transaction is rolled back
- ▶ For *every update performed* by the operation, *undo information* — in the form of the *old value in the physical log records* — is available in the log.
- ▶ The *physical log records* will be used to roll back the incomplete operation.



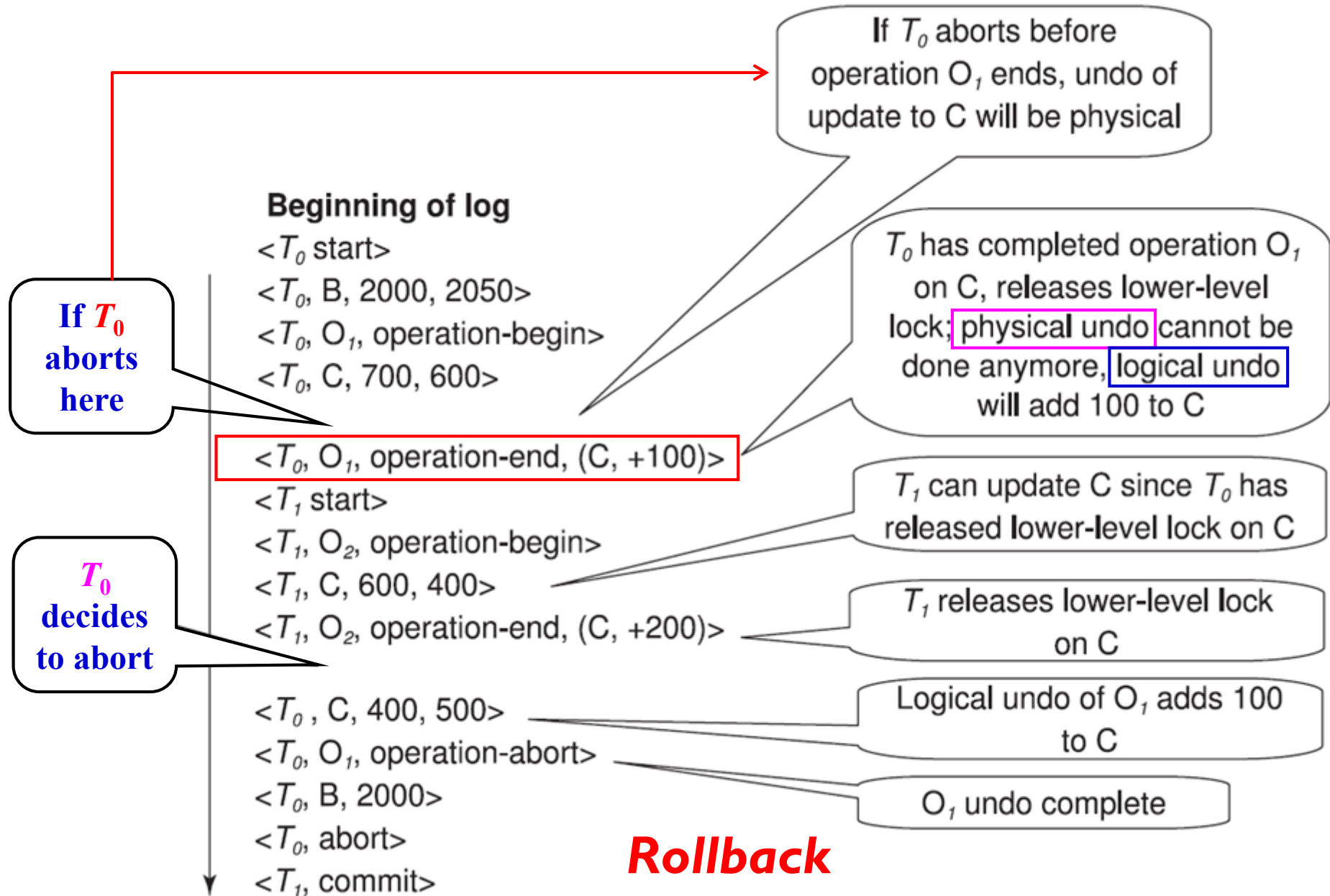
Transaction Rollback With Logical Undo

- ▶ Suppose *an operation undo was in progress* when the *system crash occurred*, which could happen if a transaction was *being rolled back* when crash occurred
- ▶ Then the *physical log records* written during operation undo would be found, and the *partial operation undo* would *itself be undone using these physical log records*.
- ▶ Continuing in the backward scan of the log, the *original operation's operation-end* record would *then be found*, and *the operation undo* would *be executed again*.
- ▶ **Note:** *Rolling back the partial effects of the earlier undo operation using the physical log records brings the database to a consistent state.*

Example

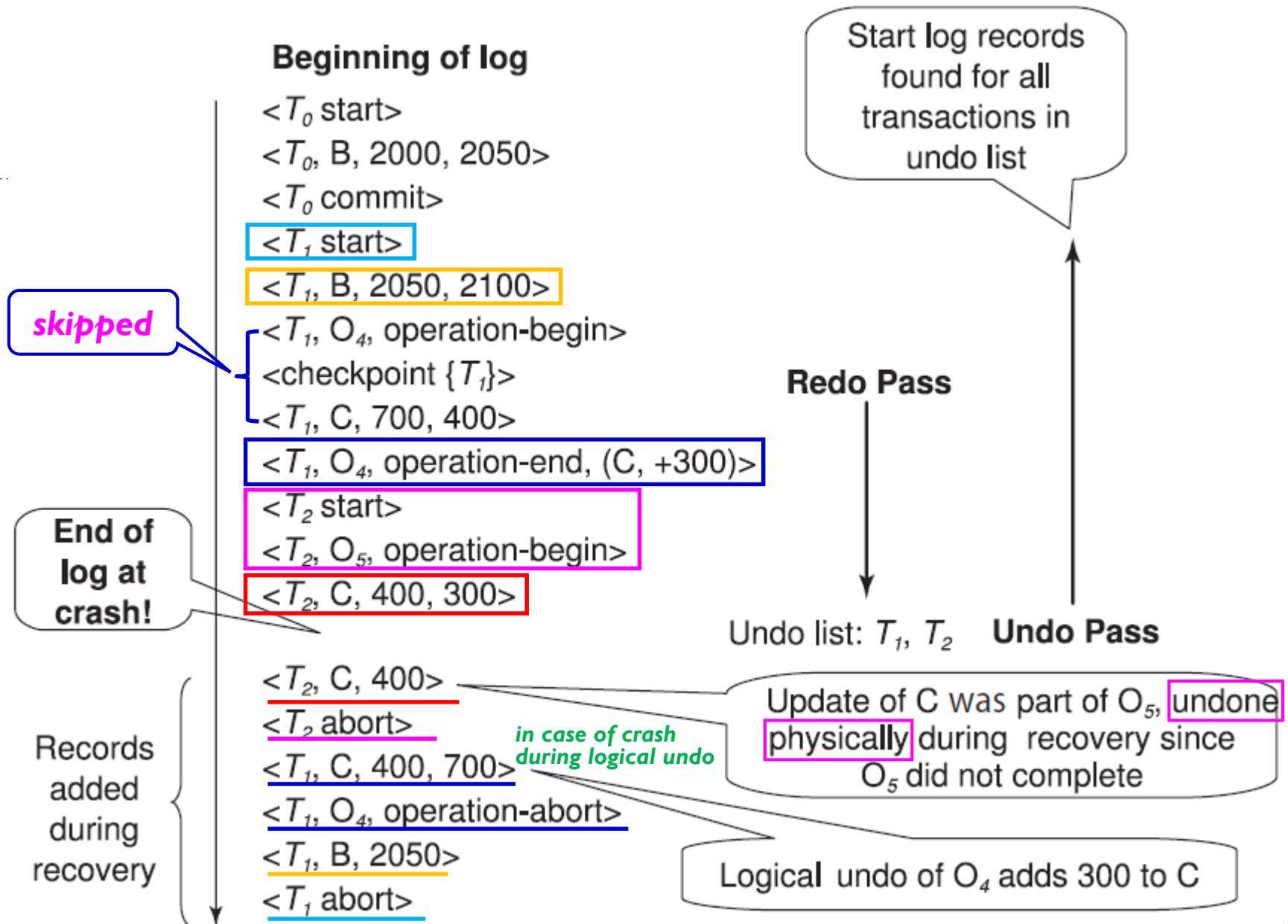
- ▶ Two transactions T_0 , T_1
 - ▶ *Add* or *subtract* a value from a data item C .
 - ▶ Operation O_1 of T_0 *subtracts 100* from the data item C .
 - ▶ Operation O_2 of T_1 *subtracts 200* from the data item C .
 - ▶ *Early lock release* on the data item C by transaction T_0 after operation O_1 completes allows transaction T_1 to update the data item using O_2 , even before T_0 completes.
 - ▶ *This necessitates logical undo.*





A log generated by T_0, T_1





Failure recovery actions with logical undo operations



Example

- ▶ **Redo Phase:** The system performs a *redo of all operations* after the *last checkpoint record* (*scanning the log forward* to the last checkpoint).
 - ▶ The actions of O_4 are redone completely, but operation O_5 was not complete.
 - ▶ The list *undo-list* initially contains T_1
 - ▶ T_2 is *added* when its $\langle T_2 \text{ start} \rangle$ is found
 - ▶ The *undo-list contains T_1 and T_2* at the end of the redo pass.



Example

- ▶ **Undo Phase:** The system scans the log *backwards from the end*
 - ▶ *The undo of operation O_5 is carried out using the old value in the physical log record.* This operation is *logged using a redo-only log record.*
 - ▶ When the operation-end record of O_4 is found, *logical undo is performed for this operation* by adding 300 to C , which is logged physically, and an operation-abort log record is added for O_4 .



Log Record Buffering

- ▶ So far, we have assumed that *every log record is output to stable storage at the time it is created*. This assumption imposing *a high overhead* on system execution:
- ▶ Output to stable storage is in *units of blocks*. In most cases, a *log record is much smaller than a block*. Thus, the output of each log record translates to *a much larger output at the physical level*.
- ▶ It is desirable to *output multiple log records at once*. To do so, we write log records to a *log buffer* in main memory.
- ▶ The *orders of log records* must be same in both buffer and stable storage.



Log Record Buffering (Cont.)

- ▶ A log record may reside *in only main memory for a considerable time* before it is output to stable storage.
- ▶ Considering system crashes, *additional requirements on the recovery techniques* must be followed to *ensure transaction atomicity*:
 1. **After** the $\langle T_i \text{ commit} \rangle$ has been *output to stable storage*, *transaction T_i enters the commit state*.
 2. **Before** the $\langle T_i \text{ commit} \rangle$ can be *output to stable storage*, *all log records pertaining to T_i must have been output to stable storage*.



Log Record Buffering (Cont.)

- ▶ Considering system crashes, *additional requirements on the recovery techniques* must be followed to *ensure transaction atomicity*:
 3. **Before** a block of data in main memory is output to nonvolatile storage, *all log records pertaining to data in that block must have been output to stable storage.*
- ▶ This is called the **write-ahead logging** (**WAL**) rule.
 - ▶ Strictly speaking, **WAL** only requires *undo information* in the log has been *output to stable storage*
 - ▶ **WAL** permits the *redo information* to be *written later*.



Log Record Buffering (Cont.)

- ▶ The three rules state situations in which *certain log records must have been output to stable storage*.
- ▶ There is **NO problem** resulting from the **output of log records earlier**, and when the system finds it necessary to output a log record to stable storage:
 - ▶ If there are *enough log records in main memory to fill a block*, it *outputs an entire block of log records*.
 - ▶ If there are *insufficient log records to fill the block*, *all log records in main memory are combined into a partially full block and are output to stable storage*.
- ▶ Writing the buffered log to disk is sometimes referred to as a **log force**.

Database Buffering

- ▶ We described the use of *a two-level storage hierarchy*.
 - ▶ The system stores the *database in nonvolatile storage* and brings *blocks of data into main memory* as needed.
 - ▶ If a *block has been modified*, it must be *output before it is overwritten*.
- ▶ The recovery algorithm we described supports the *no-force policy*: *allows a transaction to commit* even if it *has modified some blocks* that have *not yet been written back to disk*.
- ▶ *Force policy*: transactions would *force-output* all modified blocks to disk *when they commit* (*expensive*).

Database Buffering (Cont.)

- ▶ The recovery algorithm also supports the **steal policy**: *blocks containing updates of uncommitted transactions can be written to disk, even before the transaction commits.*
- ▶ **No-steal policy**: *blocks modified by a transaction that is still active should not be written to disk.*



Database Buffering (Cont.)

- ▶ As long as the *write-ahead logging rule* is followed, all the recovery algorithms we study in the chapter *work correctly even with the steal policy*.
- ▶ Further, the *no-steal policy* does not work with transactions that *perform a large number of updates*.
- ▶ Because the buffer may *get filled with updated pages* that cannot be evicted to disk, and the transaction cannot then proceed.
- ▶ As a result, the *standard approach* taken by most systems is the *steal policy*.



Database Buffering (Cont.)

Blocks modified by a transaction that is **still active** should **not be written to disk**

The system **writes modified blocks to disk** even if the **transactions** that made those modifications have **not all committed**.

No Steal

Steal

Transactions would **force-output all modified blocks** to **disk** when they **commit**

Force

No Force

Trivial

Desired

A transaction commits even if it has **modified some blocks** that have **not yet been written back to disk**



Database Buffering (Cont.)

- ▶ To illustrate the need for the *write-ahead logging requirement*, consider our banking example. Suppose the state of the *log* is:

$\langle T_0 \text{ start} \rangle$

$\langle T_0, A, 1000, 950 \rangle$

and then T_0 issues a $\text{read}(B)$.

- ▶ Assume that *the block on which B resides is not in main memory*, and that *main memory is full*.



Database Buffering (Cont.)

- ▶ Suppose that the *block on which A resides is chosen to be output to disk*.
- ▶ If the system outputs this block to disk and then *a crash occurs*, the values in the database for accounts $A=\$950$, $B=\$2000$, $C=\$700$. This database state is *inconsistent*.
- ▶ However, because of the *WAL* requirements, the *log record $\langle T_0, A, 1000, 950 \rangle$* must be output to stable storage *prior to output* of the block on which A resides.
- ▶ The system can use the log record during recovery to bring the database back to a *consistent state* ($A=1000$).



Database Buffering (Cont.)

- ▶ When a block B_1 is to be output to disk, *all log records pertaining to data in B_1* must be output to stable storage first.
- ▶ *No updates should be in progress on the block B_1* when it is output to disk. It can be ensured as follows:
 - ▶ Before a transaction performs a write on a data item, it acquires *an exclusive lock* on the block in which the data item resides.
 - ▶ Lock can *be released immediately after the update has been performed*.
 - ▶ Such locks held for short duration are called *latches (闷锁)*.

Database Buffering (Cont.)

- ▶ *To output a block to disk*
 1. First acquire an *exclusive latch* on the block (ensures no update can be in progress on the block).
 2. Then *output log records to stable storage* until all log records pertaining to block B_1 have been output.
 3. Then *output the block B_1 to disk*.
 4. Finally *release the latch on the block* output has completed.
- ▶ **Note:** *Locks on buffer* blocks are *unrelated* to *locks used for concurrency-control of transactions*, and releasing them in a non-two-phase manner *does not have any implications on transaction serializability*.



Database Buffering (Cont.)

- ▶ *Locks on buffer blocks* can also be used to *ensure that buffer blocks are not updated*, and *log records are not generated*, while a checkpoint is in progress.
- ▶ This restriction may be enforced by acquiring *exclusive locks* on *all buffer blocks*, as well as an *exclusive lock* on the *log*, before the checkpoint operation is performed.
- ▶ These locks can be released as soon as the *checkpoint operation has completed*.



Database Buffering (Cont.)

- ▶ Database systems usually have a *process* that *continually cycles through the buffer blocks*, outputting modified buffer blocks back to disk.
- ▶ The above *locking protocol* must be followed when the blocks are output.
- ▶ As a result of *continuous output of modified blocks*, the number of *dirty blocks* in the buffer is minimized.
- ▶ *Dirty blocks*: blocks that have been modified in the buffer but have not been subsequently output.



Database Buffering (Cont.)

- ▶ Therefore, the *number of blocks* that have to be output during a checkpoint is *minimized*.
- ▶ Further, when a block needs to be *evicted* from the buffer it is likely that there will be a *non-dirty block* available for eviction, allowing the input to *proceed immediately* instead of *waiting for an output* to complete.



Operating System Role in Buffer Management

- ▶ *Database buffer* can be implemented in
 - ▶ *part of real main-memory* (reserved and managed by the database).
 - ▶ *virtual memory* (provided by OS).
- ▶ Implementing buffer in reserved main-memory has *drawbacks*:
 - ▶ *limiting flexibility*: The buffer must be kept small enough that other applications have sufficient main memory available for their needs.
 - ▶ Even when the other applications are not running, the database will *not be able to make use of all the available memory*.

Operating System Role in Buffer Management

- ▶ If *database buffers* are generally implemented in *virtual memory*.
- ▶ The OS knows about *the memory requirements* of all processes in the system, ideally it should be in charge of deciding *what buffer blocks must be force-output to disk, and when*.
- ▶ But, to ensure the *write-ahead logging requirements*, the OS should not write out the database buffer pages itself, but instead should *request the database system to force-output the buffer* blocks (*write log first*).



Operating System Role in Buffer Management

- ▶ If *database buffers* are generally implemented in *virtual memory*.
- ▶ Unfortunately, almost all current-generation OSs retain *complete control* of virtual memory. The OS reserves *space on disk for storing virtual-memory pages* that are not currently in main memory (*swap space*).
- ▶ If the operating system decides to output a block B_x , that block is output to the *swap space* on disk, and there is no way for the DB to get control of the output of buffer blocks.



Operating System Role in Buffer Management

- ▶ If *database buffers* are generally implemented in *virtual memory*.
 - ▶ Therefore, if the database buffer is in virtual memory, transfers between database files and the buffer in virtual memory *must be managed by the database system*.
 - ▶ This approach may result in extra output of data to disk.
 - ▶ If *a block B_x is output by the operating system*, that block is not output to the database. Instead, it is output to the swap space for the operating system's virtual memory.
 - ▶ When the database system needs to output B_x , the operating system may need first to input B_x from its swap space.



Operating System Role in Buffer Management

- ▶ If *database buffers* are generally implemented in *virtual memory*.
 - ▶ Therefore, if the database buffer is in virtual memory, transfers between database files and the buffer in virtual memory *must be managed by the database system*.
 - ▶ This approach may result in extra output of data to disk.
 - ▶ Thus, instead of **a single output of B_x** , there maybe *two outputs of B_x* (one by the operating system, and one by the database system) and one extra input of B_x .



Fuzzy Checkpointing

- ▶ As we know, the *checkpointing technique* requires that all updates to the database be *temporarily suspended* while the *checkpoint is in progress*.
- ▶ To avoid *long interruption*, we can permit updates to start *once the checkpoint record has been written*, but *before the modified buffer blocks are written to disk* (such modified *checkpoint* is called a *fuzzy checkpoint*).



Fuzzy Checkpointing (Cont.)

- ▶ Since pages are output to disk only *after* the checkpoint record has been written, it is possible that the system could *crash before all pages are written*. Thus, *a checkpoint* on disk may be *incomplete*.
- ▶ One way to deal with incomplete checkpoints is this
 - ▶ The *location in the log* of the last completed checkpoint is stored in a fixed position, *last-checkpoint*, on disk.
 - ▶ The system does *not* update this information when it writes the checkpoint record. Instead, before it writes the checkpoint record, it creates a list of all modified buffer blocks.



Fuzzy Checkpointing (Cont.)

- ▶ Since pages are output to disk only *after* the checkpoint record has been written, it is possible that the system could *crash before all pages are written*. Thus, *a checkpoint* on disk may be *incomplete*.
- ▶ One way to deal with incomplete checkpoints is this:
 - ▶ The last-checkpoint information is updated *only after all buffer blocks in the list* of modified buffer blocks have been output to disk.



Failure with Loss of Nonvolatile Storage

- ▶ So far we assumed *no loss of non-volatile storage*, we need to be prepared to deal with this type of failure.
- ▶ The basic scheme to deal with loss of non-volatile storage is: *periodically dump the entire content of the database to stable storage*
- ▶ If a failure occurs, the system uses *the most recent dump* (转储) in restoring the database to a *previous consistent state*.
- ▶ Once this restoration has been accomplished, the system uses the *log* to bring the database system to the *most recent consistent state*.



Failure with Loss of Nonvolatile Storage

- ▶ One approach to database dumping requires that ***no transaction may be active during the dump procedure***, and uses a procedure similar to ***checkpointing***:
 - ▶ Output ***all log records*** currently residing in main memory onto ***stable storage***.
 - ▶ Output ***all buffer blocks*** onto the ***disk***.
 - ▶ Copy the ***contents of the database*** to ***stable storage***.
 - ▶ Output ***a record <dump>*** to ***log on stable storage***.



The End of Database Course

Thanks

